

Internship Screening Assignment — Level 1

RAG over a Hindi Document

MegaMindz AI Solutions Pvt Ltd · Talent Team · 2026

1. Overview

At MegaMindz AI Solutions, much of our work involves building Retrieval-Augmented Generation (RAG) systems over Indian-language content. This assignment is the **first level of screening** for our AI/ML internship.

You will build a small, working RAG pipeline that answers questions about a **Hindi-language document**. We care less about a polished product and more about **how you think** — your choices around chunking, embeddings, vector storage, and handling Devanagari text.

Expected effort: 6–10 hours. **Deadline:** _____ (5 days from receipt).

We provide the document for you. Use the supplied file

Agni_Ki_Udaan_Adhyayan_Sahayika_Hindi.pdf — a self-contained Hindi document on the life of A.P.J. Abdul Kalam (narrative text, plus timeline and awards tables). Everyone works on the same document so we can evaluate fairly. You do *not* need to find your own source.

2. What You Will Build

1. Take the supplied **Hindi-language document** (Agni_Ki_Udaan_Adhyayan_Sahayika_Hindi.pdf).
2. **Chunk** the document and store the **embeddings** in an open-source vector database — **ChromaDB** or **Qdrant**.
3. Given a set of **queries (Hindi or English)**, **retrieve** the most relevant chunks.
4. **Generate a final answer** using any LLM you have access to.

3. Step-by-Step Instructions

1. **Load & extract** the text (and, ideally, the tables) from the supplied PDF.
2. **Clean & normalize** the Devanagari text (see §7).
3. **Chunk** the document — decide fixed-size, sentence/paragraph, or token-aware with overlap. Be ready to justify chunk size and overlap.
4. **Embed & store** chunks + embeddings + metadata in ChromaDB or Qdrant using a Hindi/multilingual embedding model.
5. **Retrieve** the top-k chunks for each test query.
6. **Generate** a grounded answer from the retrieved chunks — based on the document, not the model's prior knowledge.

7. **Cite your sources.** Every answer must state **where it came from** — the retrieved chunk(s) and their metadata (at minimum the **page number**, plus any section/heading and chunk ID you store). An answer with no citation is not accepted.
8. **Write the README** (see §8).

4. Test Queries You Must Answer

Run **at least these 6 queries** through your pipeline and include the answers your system produces. (You may add more.) They are answerable directly from the document.

#	Query	Language
1	SLV-III ने किस उपग्रह को कक्षा में स्थापित किया और किस वर्ष?	Hindi
2	कलाम को भारत रत्न किस वर्ष प्राप्त हुआ?	Hindi
3	पोखरण-II में कलाम की क्या भूमिका थी?	Hindi
4	Which institution did Kalam attend to study aeronautical engineering?	English
5	Who co-wrote the autobiography, and in what year was it published?	English
6	How and where did Kalam die in 2015?	English

Cross-lingual matters: English queries (4–6) must still retrieve the correct *Hindi* chunks. That is the point of using a multilingual embedding model.

Required output format — answer + citation

For every query, print the answer **together with the source it came from**. Carry the **page number** as chunk metadata during ingestion and surface it at answer time. A minimal acceptable format:

Field	Example
Query	कलाम को भारत रत्न किस वर्ष प्राप्त हुआ?
Answer	1997
Source (citation)	page: 20 · section: "पुरस्कार एवं सम्मान" · chunk_id: 47 · score: 0.83

Store page number, section/heading, and a chunk ID as metadata in your vector DB so these citations are produced automatically — not hand-written after the fact.

5. Tools & Models — What You May Use

- **Vector DB (required):** ChromaDB or Qdrant (open-source, runs locally).
- **Embeddings:** any multilingual model — e.g. `intfloat/multilingual-e5`, `paraphrase-multilingual-*`, BGE-m3, or a hosted embedding API.
- **LLM:** anything you have access to — a hosted API (Claude, GPT, Gemini) or a local model (Llama, Qwen, etc.).
- **Language:** Python preferred; any language is fine if it runs.

You will **not** be judged on having paid API access. A clean pipeline with a small local model beats a messy one with a frontier model.

6. Deliverables

Submit a single Git repository (GitHub/GitLab) or a zip archive containing:

1. **All source code** — runnable end to end.
2. A **requirements.txt** (or equivalent) to reproduce your environment.
3. The **list of test queries** (§4) and the **answers your system produced, each with its source citation** (page + metadata, per §4) — a text/markdown log or notebook output is fine.
4. A short **README.md** (see §8).

7. Handling Hindi Text — What We Look For

- **Unicode normalization** (e.g. NFC) so visually-identical text matches.
- **Multilingual / Hindi-aware embeddings** — English-only models retrieve poorly on Hindi.
- **Sensible chunk boundaries** — respect the Hindi danda `|`, not just `.`.
- **Cross-lingual retrieval** — an English query finds the right Hindi chunk.
- **Correct encoding** — UTF-8 throughout, no mojibake, tables extracted readably.

8. README — Required Contents (≈ 1 page)

1. Your **chunking strategy** — chunk size, overlap, and *why*.
2. Why you picked **Chroma or Qdrant**.
3. How you **handled the Hindi text** — embeddings, normalization, segmentation.
4. What you'd **improve with more time** — honest limitations and next steps.
5. **How to run it** — exact commands to reproduce your results.

9. Evaluation Criteria (scored out of 100)

#	Criterion	Weight	What "good" looks like
1	It runs	20	Installs and runs from your README without guesswork; produces answers end to end.
2	Retrieval quality	20	Retrieved chunks are actually relevant; cross-lingual (EN→HI) retrieval works.
3	Hindi handling	15	Correct encoding, normalization, sensible segmentation, multilingual embeddings.
4	Answer quality	10	Answers grounded in retrieved context, not hallucinated.
5	Citations & source traceability	10	Every answer cites its source — correct page number and metadata (section, chunk ID) stored in the vector DB and surfaced automatically, not hand-written.
6	Design choices	10	Thoughtful, justified decisions — not tutorial defaults.
7	Code quality	10	Readable, organized, sensible structure; not over-engineered.
8	README & communication	5	Clear reasoning, honest about limitations.

Bonus (up to +10, capped at 100): a retrieval evaluation (is the correct chunk in top-k?); a simple UI/CLI; metadata filtering, re-ranking, or hybrid (keyword + vector) search; correct extraction of the **tables** from the document.

10. Submission Checklist

- ☐ Code runs end to end from a clean environment following my README.
- ☐ `requirements.txt` (or equivalent) included and complete.
- ☐ All 6 test queries answered, with the output produced.
- ☐ Every answer includes its **source citation** — page number + metadata.
- ☐ README covers chunking, DB choice, Hindi handling, improvements, and run steps.
- ☐ No hardcoded secrets / API keys committed to the repo.
- ☐ Repo/zip link works and is accessible to us.

11. How to Submit & Ground Rules

Push to a **public GitHub/GitLab repo** and share the link, or email a zip to _____. Subject line: RAG Assignment – <Your Name> .

You may use AI assistants and reference docs (we expect you to), but you must **understand and be able to explain every part** of your submission. Shortlisted candidates will walk us through their code in a short follow-up call. Focus on a working, well-reasoned pipeline — not a large or fancy one. **Good luck!**

